

Early work in entity disambiguation typically used handcrafted features to map entity mentions to entity definitions. The task of entity disambiguation can be formally defined as follows: Given entity definitions from a knowledge base, and a text document annotated with entity mentions, the task is to map each entity mention to its corresponding entity definition including *<None>*. This can be treated as a multi-class classification problem modelled using supervised learning techniques, and can be addressed using classical machine learning techniques with handcrafted features. Of late, neural network based techniques have typically been used for entity disambiguation with recurrent neural networks (LSTMs) and transformer based encoders to capture the representation of the entity mention and entity definition. This learnt representation can be fed to a standard MLP classifier for the final classification.

Once entity mentions have been disambiguated to the corresponding definition, the idea is to learn the relation that exists between a pair of entities using the unstructured text. Consider the example we discussed earlier: 'The president was born in Hawaii'. Once the entity mention 'president' has been disambiguated to 'Barack Obama', as in the previous example, we are interested in identifying the relation present in the sentence containing the entity mention. In this case, it is straightforward as the predicate argument 'born' directly provides the relation. This allows us to extract the tuple *<Barack Obama, 'Born', Hawaii>* where Barack Obama and Hawaii are the entity definitions and the relation between these two entities is 'born'.

We know that entities have specific types. Relation classes, in fact, require that entities involved in the relation should be of certain types. In the above example, it is clear that the first entity should be a person. The second entity in this case is a geographical place. On the other hand, we can have a sentence 'The President was born in 1960', in which the second entity is of type *<DATE>*.

The fact that specific relation types require specific entity types is interesting, because it allows us to use the relation (if it is known) to disambiguate the entity mentioned. Hence there is considerable interaction between relation extraction (relation classification) and entity disambiguation. Therefore, instead of treating relation extraction and entity disambiguation as separate problems, a few works are now exploring these as a joint learning problem.

As we mentioned before, neural approaches are becoming popular in both entity disambiguation and relation extraction. Typical approaches include sentence representation using word embeddings, followed by a bidirectional LSTM layer (one or multiple layers stacked over each other), and a MLP classifier which uses the contextual representations from biLSTM layers to make the final prediction using softmax. Non-neural representations in earlier works used handcrafted features and also the dependency parse features between words in the sentence. Dependency parse related features enabled modelling of syntactic information. However, with the advent of neural approaches, explicit modelling of syntactic information was not considered necessary since the representations may implicitly learn syntactic features.

The issue with neural approaches was in handling long sentences, where the relation between two entity mentions was spread over a longer dependency path. This long dependency path became difficult for LSTMs to effectively capture in their representation, leading to poor performance in such sentences. Hence, there arose a need to reintroduce syntactic information in neural approaches for relation extraction.

The arrival of graph neural networks, namely, graph convolutional networks (GCNs) and graph attention networks (GATs) enabled leveraging them to add syntactic information in neural approaches for relation extraction. The key addition to a standard neural method for relation extraction is to add a GCN layer after the biLSTM layer. The GCN layer takes as inputs the biLSTM outputs and the dependency parse tree from an external dependency parser of the original input sentence. The outputs from the GCN layer are passed to the MLP classifier for the final classification. Each node in GCN is the biLSTM representation corresponding to each token, and the dependency parse tree is used to construct the graph between these nodes.

Note that two words that are wide apart in the linear sentence form are brought together by a direct dependency edge in their parse tree representation. Hence, adding the GCN layer enables the relation extraction model to handle the long-term dependencies easily. One thing to note is that standard GCNs represent undirected graphs, whereas graphs from dependency parse trees are directed. While it is possible to treat the dependency parse tree graph to be undirected, it results in loss of information. Hence, GCNs can be extended to handle directed edges by creating two edges with different labels (one between *u, v* and another between *v, u*). Also note that dependency parse trees contain labels for different edges. Hence, we need to use different parameter weights corresponding to different labels. However, this results in an increased number of parameters to be learnt. Often, systems employing GCN for representing syntactic representation avoid this parameter explosion by using different parameter matrices for different directions and not considering the label information. Only bias terms in the network representation are driven by the label information. This prevents over-parameterisation of the network. We will talk more about using GATs for relation extraction and entity disambiguation in our next column.

Feel free to reach out to me over LinkedIn/email if you need any help in your coding interview preparation. If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Wishing all our readers a very happy new year 2022! Stay healthy and stay safe. **END** 🐧

 **By: Sandya Mannarswamy**

The author is an expert in natural language processing and is currently working as an independent researcher. Her interests include natural language processing, machine learning and AI.